CSA02 C Programming – CO4 Assignment Answers

Assignment 4: Structures, Unions and Storage Classes

Note: These are model answers based on the uploaded assignment sheets. Replace student-specific details and add your own compiler screenshots, timing measurements and GitHub links before submission.

Task 1 – Patient register

Defects: parallel arrays can become misaligned; char phone[10] cannot hold ten digits plus NUL; float cannot represent systolic/diastolic pair; date text is unvalidated; count has no capacity check; globals create coupling; sorting requires coordinated swaps. Use nested structures.

```
typedef struct{int day,month,year;} Date;
typedef struct{int systolic,diastolic;} BP;
typedef struct{char name[50];int age;char phone[16];BP pressure;Date visit;} Patient;
int addPatient(Patient *a,int *count,int cap,const Patient *p){if(*count>=cap)return 0;a[*count]=*p;++*count;return 1;}
```

Sorting swaps one Patient record; parallel arrays can be efficient for column-wise processing but every column must be moved together. Check sizeof for padding.

Task 2 – Courier parcels

Entities: Date, Address, Person, Parcel. Parcel contains ID, weight, booking date, sender, receiver, service and charge; Person nests Address. Sort by weight with selection sort; city search scans all records. PIN expressions: list[2].receiver.address.pincode and p->receiver.address.pincode.

```
typedef struct{int day,month,year;} Date;
typedef struct{char city[20];int pincode;} Address;
typedef struct{char name[40];Address address;} Person;
typedef struct{int id;float weight;Date booked;Person sender,receiver;int express;long charge;} Parcel;
void sort_by_weight(Parcel p[],int n){for(int i=0;i<n-1;i++){int m=i;for(int j=i+1;j<n;j++)if(p[j].weight<p[m].weight)m=j;if(m!=
```

Selection sort is O(n²), with at most n−1 swaps of sizeof(Parcel) bytes. Flat structures shorten access but duplicate address layout and reduce reuse.

Task 3 – Tagged union

All-fields structure likely uses 16 bytes on a common ABI: float 4 + int 4 + char[8]. Tagged union typically uses enum tag 4 + union 8 = 12 bytes; confirm with sizeof because ABI alignment varies. Reading an inactive union member reinterprets bytes and is not a valid conversion.

```
void show(const struct reading *r){switch(r->type){case TEMPERATURE:printf("Temp %.1f C\\n",r->value.celsius);break;case HUMIDIT
```

Separate typed arrays save per-record tags but require multiple arrays and order metadata. void* allocations add pointer/allocation overhead. Tagged union offers O(1) access and compact fixed-size records.

Task 4 – Structure debugging

Pass-by-value changes only a local copy; main's balances remain unchanged. p is a pointer, so p.number is invalid; use p->number. Addresses printed inside and outside prove the copy differs.

```
void addInterest(struct account *a,float rate){a->balance += a->balance*rate/100.0f;}
int main(void){struct account list[3]={{101,5000},{102,12000},{103,800}};for(int i=0;i<3;i++)addInterest(&list[i],4);struct acco
```

New balances: 5200, 12480, 832. Pass by value for deliberate copy; pointer to modify; const pointer for large read-only records. Returning a modified copy is valid but copies the structure.

Task 5 – Storage classes

Automatic local token resets on every call: 1001 1001 1001. Static local persists: 1001 1002 1003. A global has program lifetime and can be declared extern in another file. register is only a hint and its address cannot be taken. Two counters are easiest when caller-owned counters are passed by pointer. Function argument evaluation order in one printf is unspecified; call functions in separate statements.

```
int next_token(void){static int token=1000;return ++token;}
```

Task 6 – Canteen design

Entities: Item common fields; Packaged subtype has expiry date; Cooked subtype has preparation time; Combo has up to three component IDs; Sale stores item, quantity and revenue. Use a Date nested structure, array of Item, and tagged union. Validate stock before updating. Expiry warning compares date values and reports packaged items within two days.

```
typedef struct{int d,m,y;} Date;
typedef enum{PACKAGED,COOKED,COMBO} Kind;
typedef struct{Date expiry;} Packaged;
typedef struct{int hour,minute;} Cooked;
```

```
typedef struct{int ids[3],count;} Combo;
typedef struct{int id;char name[40];long price;int stock;Kind kind;union{Packaged p;Cooked c;Combo combo;} detail;} Item;
typedef struct{int item_id,qty;long revenue;} Sale;
int record_sale(Item *it,int qty,Sale *s){if(qty<=0||qty>it->stock)return 0;it->stock-=qty;s->item_id=it->id;s->qty=qty;s->rever
```

Expiry pseudocode: for each packaged item, calculate days to expiry; if 0<=days<=2, warn. Sort Sale records by revenue descending. Array search/update O(n); qsort O(s log s). Linked list insertion/removal at a known node O(1), search O(n), with extra next-pointer and allocation overhead.